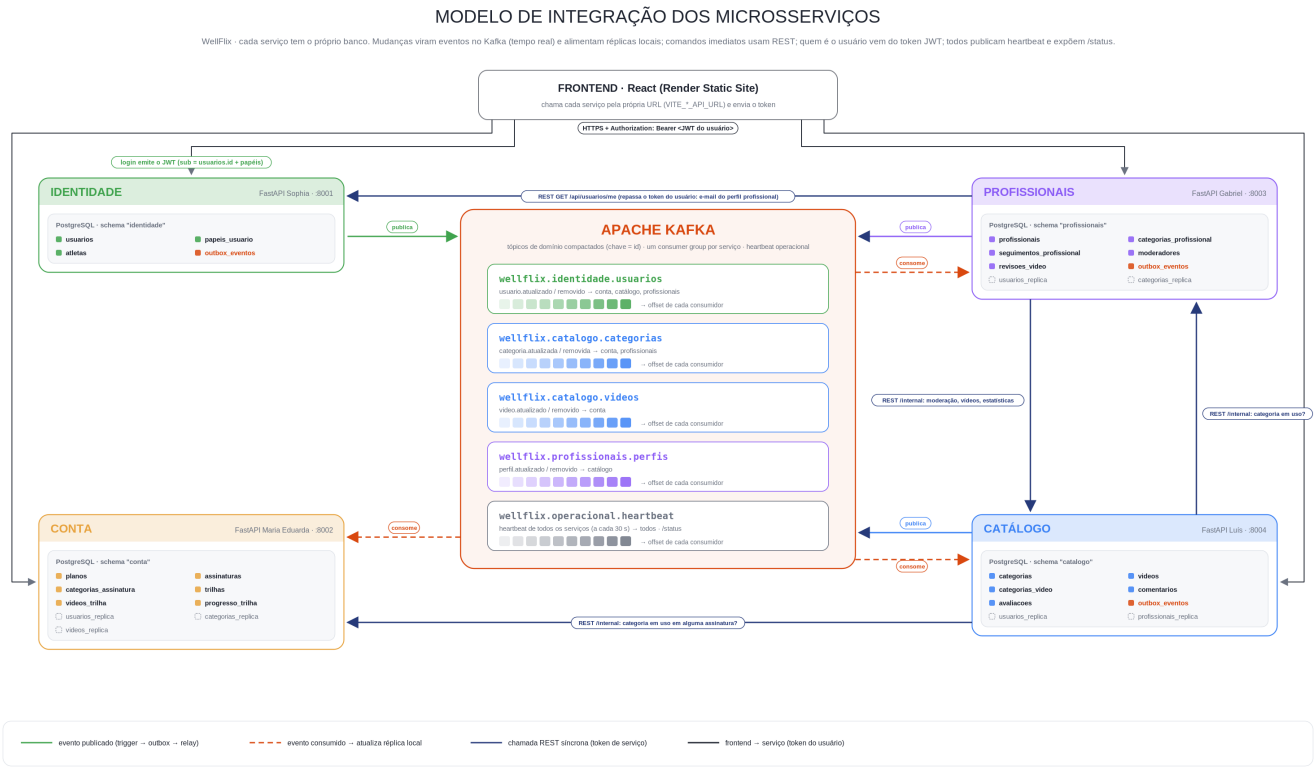
Modelo arquitetônico de integração — WellFlix

Grupo 2: Sophia (identidade), Maria Eduarda (conta), Gabriel (profissionais e moderação) e Luís (catálogo).
Referência: capítulo 17 (Microservices Architecture) de Fundamentals of Software Architecture, de Mark Richards e Neal Ford.

Como o sistema está dividido

O WellFlix é formado por quatro microsserviços, um para cada parte do negócio, e cada integrante cuida de um deles. A **identidade** cuida das contas e do login. A **conta** cuida de planos, assinaturas e trilhas. O serviço de **profissionais** cuida dos perfis, dos seguidores e da moderação. O **catálogo** guarda categorias, vídeos, comentários e avaliações.

Seguindo a ideia de contexto delimitado do livro, cada serviço tem o próprio banco: um schema separado no PostgreSQL, que nenhum outro serviço acessa. Quando um serviço precisa de um dado que pertence a outro (o nome de um usuário, por exemplo), ele guarda uma cópia local, chamada réplica. O livro chama essa escolha de "preferir duplicação a acoplamento": é melhor repetir uma informação do que fazer todos os serviços dependerem do mesmo banco.



Como os serviços conversam

O capítulo descreve duas formas de comunicação, e usamos as duas.

Por eventos (assíncrona). Quando algo muda (um usuário se cadastra, uma categoria é criada, um vídeo é aprovado), o serviço dono do dado publica um evento no **Apache Kafka**. Os serviços interessados recebem o evento e atualizam suas réplicas. Ninguém coordena esse processo; cada serviço reage por conta própria. O livro chama esse modelo de **coreografia** e o recomenda como padrão para microsserviços, porque mantém os serviços desacoplados.

Por REST (síncrona). Algumas ações precisam de resposta na hora. Quando um moderador aprova um vídeo, por exemplo, o serviço de profissionais chama diretamente o catálogo para mudar a situação do vídeo. Essas chamadas usam rotas internas que só aceitam um token próprio entre serviços.

Para não perder eventos, usamos o padrão transactional outbox: o evento é gravado no banco na mesma transação do dado, e só depois um processo o publica no Kafka. Assim um evento nunca é publicado sem o dado existir, e um dado nunca é gravado sem gerar seu evento. Se o Kafka cair, os eventos esperam no banco e são enviados quando ele volta. Isso também resolve a questão de transações entre serviços, que o livro recomenda evitar: nenhuma operação precisa de uma transação que envolva mais de um banco.

Por que Kafka e não RabbitMQ

Os dois são brokers de mensagens, mas funcionam de jeitos diferentes. O RabbitMQ trabalha com filas: depois de lida, a mensagem some. O Kafka guarda os eventos como um histórico, e cada serviço lê no seu ritmo. Como o nosso principal uso é manter réplicas atualizadas, o Kafka se encaixa melhor. Ele guarda o último estado de cada usuário, categoria, vídeo e perfil, e uma réplica pode ser reconstruída do zero relendo esse histórico.

Usamos cinco tópicos:

Tópico	Quem publica	Quem consome
`wellflix.identidade.usuarios`	identidade	conta, catálogo, profissionais
`wellflix.catalogo.categorias`	catálogo	conta, profissionais
`wellflix.catalogo.videos`	catálogo	conta
`wellflix.profissionais.perfis`	profissionais	catálogo
`wellflix.operacional.heartbeat`	todos	todos

Dados sensíveis, como e-mail, CPF e senha, nunca saem do serviço de identidade.

Monitoramento comum aos quatro serviços

O livro sugere que preocupações operacionais (logs, monitoramento, verificação de saúde) sejam resolvidas por um componente comum, o sidecar, sem misturar regras de negócio. Fizemos isso com um módulo compartilhado (`observabilidade.py`) que os quatro serviços usam do mesmo jeito:

- **Logs no mesmo formato**, com linhas que mostram cada evento PUBLICADO e CONSUMIDO e cada chamada REST entre serviços.
- **Um identificador por ação** (`X-Request-ID`), que acompanha a requisição pelas chamadas REST e pelos eventos. Assim dá para seguir uma mesma ação nos logs de todos os serviços por onde ela passa.
- **Heartbeat**: a cada 30 segundos, cada serviço publica no Kafka um sinal de que está vivo e escuta o sinal dos outros.
- **Rota `/status`**, que mostra se o banco e o broker estão funcionando, quantos eventos foram publicados e consumidos e quando chegou o último heartbeat de cada serviço.

Publicação no Render

Cada integrante publica seu serviço no próprio workspace do Render, a partir da branch `main`. O frontend em React também está no Render. O banco é o PostgreSQL do Labsoft, com um schema por serviço. Como o Render não oferece Kafka, criamos um Kafka gerenciado no Aiven.

Serviço	Endereço
Frontend	https://wellflix-frontend.onrender.com
identidade	https://wellflix-identidade-sophia.onrender.com

Serviço	Endereço
conta	https://wellflix-conta-maria.onrender.com
profissionais	https://wellflix-profissionais-moderadores.onrender.com
catálogo	https://wellflix-catalogo-luis.onrender.com

O que precisou ser corrigido. Na primeira verificação, os serviços estavam no ar, mas toda rota que usava o banco dava erro 500. O log mostrava que as tabelas não existiam: o código novo tinha sido publicado, mas os schemas nunca tinham sido criados no banco. Mudamos os serviços para que cada um crie e atualize o próprio schema ao iniciar. Também adaptamos o código para receber os certificados do Kafka por variável de ambiente e criamos um script (`configurar_render.py`) que configura os quatro serviços de uma vez.

Resultados

Testamos o sistema primeiro na máquina local (com PostgreSQL e Kafka no Docker) e depois no Render. O teste é um script que percorre um fluxo de ponta a ponta, em que cada passo só funciona se a integração estiver ativa:

1. um atleta se cadastra na identidade;
2. o moderador cria uma categoria no catálogo;
3. um profissional se autoriza nessa categoria;
4. o profissional envia um vídeo;
5. o moderador aprova o vídeo;
6. o atleta coloca o vídeo numa trilha.

Os seis passos funcionaram nos dois ambientes. No Render, os quatro serviços se conectaram ao Kafka e passaram a receber o heartbeat uns dos outros. Ao carregar os dados de demonstração, foram gerados 46 eventos, e todos foram consumidos pelos serviços certos, sem nenhum erro nos logs.

O trecho abaixo, tirado dos logs do Render, mostra a aprovação de um vídeo. O identificador `demo-5-c8891a` aparece em três serviços diferentes. O serviço de profissionais chama o catálogo por REST. O catálogo publica o evento no Kafka, e a conta o recebe cerca de 1 segundo depois. Logo em seguida, o atleta consegue adicionar o vídeo à trilha:

```
20:07:38.517 | profissionais | rest    | [req=demo-5-c8891a] REST profissionais -> catalogo POST /internal/
videos/4f996759-…/moderacao -> 200
20:07:39.260 | catalogo      | eventos | [req=demo-5-c8891a] PUBLICADO video.atualizado ->
wellflix.catalogo.videos
20:07:39.865 | conta         | eventos | [req=demo-5-c8891a] CONSUMIDO video.atualizado de catalogo (997 ms após
a publicação)
20:07:42.816 | conta         | http    | [req=demo-6-8b1311] POST /api/trilhas/c38bb85b-…/videos -> 201
```

E aqui, o heartbeat: cada serviço recebendo o sinal dos outros pelo Kafka.

```
19:58:49.424 | catalogo      | heartbeat | HEARTBEAT recebido de identidade #11 (atraso 1943 ms)
19:58:49.955 | catalogo      | heartbeat | HEARTBEAT recebido de conta #11 (atraso 1850 ms)
19:58:57.182 | catalogo      | heartbeat | HEARTBEAT recebido de profissionais #11 (atraso 1864 ms)
```

Os logs completos estão nas pastas `logs-integracao/` (execução local) e `logs-render/` (produção).

Limitações

- **Os dados não aparecem em todos os serviços no mesmo instante.** Uma réplica leva de alguns milissegundos (local) a alguns segundos (Render) para refletir uma mudança. É o custo de manter os bancos separados.
- **O plano gratuito do Render "desliga" serviços parados** depois de cerca de 15 minutos. Nenhum evento se perde: ao voltar, o serviço continua de onde parou. Mas convém abrir o site alguns minutos antes de uma demonstração.
- **Não usamos API gateway.** O frontend chama cada serviço diretamente. O livro trata o gateway como opcional, e ele poderia ser adicionado depois sem mudar os serviços.
- **O Kafka está no período de avaliação do Aiven** (30 dias), o que basta para esta entrega.